

Software Development and AI Concepts

1. SOLID Principles (Basic)

Introduction

SOLID is a set of five object-oriented design principles that help developers write clean, maintainable, scalable, and reusable code. These principles were introduced by software engineer Robert C. Martin (Uncle Bob).

Following SOLID principles makes software easier to understand, test, and modify.

S – Single Responsibility Principle (SRP)

This principle states that a class should have only one responsibility or one reason to change.

Example

Suppose we have a Student class.

Bad Design:

- Stores student data.
- Generates reports.
- Sends emails.

Good Design:

- Student Class → Stores student information.
- Report Class → Generates reports.
- Email Class → Sends emails.

Benefit

Code becomes easier to maintain and modify.

O – Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification.

This means we should be able to add new functionality without changing existing code.

Example

A payment system currently supports Credit Card payments.

Instead of modifying the existing code every time a new payment method is added, create separate classes for:

- Credit Card Payment
- UPI Payment
- PayPal Payment

Benefit

Reduces bugs and improves scalability.

L – Liskov Substitution Principle (LSP)

Objects of a derived class should be replaceable with objects of the base class without affecting program behavior.

Example

If Bird is a parent class, a Sparrow can replace Bird without causing issues.

Benefit

Improves inheritance design.

I – Interface Segregation Principle (ISP)

Clients should not be forced to depend on methods they do not use.

Instead of creating one large interface, create smaller and more specific interfaces.

Benefit

Reduces unnecessary dependencies.

D – Dependency Inversion Principle (DIP)

High-level modules should depend on abstractions rather than concrete implementations.

Example

Instead of directly depending on a database class, depend on a database interface.

Benefit

Makes applications more flexible and easier to test.

Why SOLID Principles Are Important

- Cleaner code
 - Easier maintenance
 - Better scalability
 - Improved testing
 - Reduced bugs
-

2. Coding Standards

Introduction

Coding standards are a set of guidelines and best practices followed while writing software code. They help developers create readable, maintainable, and consistent code.

Common Coding Standards

Meaningful Variable Names

Bad:

```
x = 100
```

Good:

```
student_age = 100
```

Proper Indentation

Correct indentation improves readability.

Example:

```
if age > 18:  
    print("Eligible")
```

Code Comments

Comments explain complex logic.

Example:

```
# Calculate total salary
salary = basic + allowance
```

Consistent Naming Conventions

Examples:

- camelCase
 - PascalCase
 - snake_case
-

Avoid Code Duplication

Reusable functions should be created instead of repeating code.

Error Handling

Applications should handle errors properly.

Example:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Benefits of Coding Standards

- Better readability
 - Easier debugging
 - Team collaboration
 - Maintainability
 - Professional code quality
-

3. Authentication Types

Introduction

Authentication verifies the identity of users before allowing access to systems, applications, or APIs.

Password Authentication

The user enters a username and password.

Example:

- Gmail Login
 - Facebook Login
-

Multi-Factor Authentication (MFA)

Requires multiple verification methods.

Examples:

- Password + OTP
- Password + Fingerprint

Benefit

Improved security.

OAuth Authentication

OAuth is an authorization framework that allows users to log in through third-party services without sharing passwords.

Examples:

- Login with Google
- Login with GitHub
- Login with Microsoft

OAuth Flow

1. User clicks Login with Google.
2. Google verifies identity.
3. User grants permission.

4. Access token is generated.
5. Application receives authorization.

Benefits

- Secure
 - Convenient
 - Widely used
-

JWT (JSON Web Token)

JWT is a token-based authentication mechanism.

After login:

1. Server verifies credentials.
2. JWT token is generated.
3. Token is sent to the client.
4. Client includes token in future requests.

Components of JWT

- Header
- Payload
- Signature

Benefits

- Stateless authentication
 - Fast performance
 - API-friendly
-

SSO (Single Sign-On)

SSO allows users to access multiple applications using one login.

Example:

A company employee logs in once and accesses:

- Email
- HR Portal
- Attendance System

without logging in again.

Benefits

- Better user experience
 - Centralized authentication
 - Reduced password management
-

4. Tokenizer in Different LLM Providers

What is a Tokenizer?

A tokenizer converts text into smaller units called tokens before sending them to a Large Language Model (LLM).

LLMs do not understand words directly. They process tokens.

Example:

Sentence:

“Artificial Intelligence is powerful.”

May be split into:

- Artificial
- Intelligence
- is
- powerful

These tokens are converted into numerical representations that the model can process.

Why Tokenization Is Important

- Controls context length.
 - Affects API cost.
 - Improves processing efficiency.
 - Helps LLM understand text.
-

Different LLM Providers and Tokenizers

OpenAI Models

Used in GPT models.

Tokenizer:

- Byte Pair Encoding (BPE)

Examples:

- GPT-4
 - GPT-4o
-

Google Models

Used in Gemini models.

Tokenizer:

- SentencePiece

Examples:

- Gemini Pro
 - Gemini Flash
-

Anthropic Models

Used in Claude models.

Tokenizer:

- Custom tokenizer optimized for Claude.

Examples:

- Claude Sonnet
 - Claude Opus
-

Meta Models

Used in Llama models.

Tokenizer:

- SentencePiece

Examples:

- Llama 3
 - Llama 4
-

Mistral Models

Tokenizer:

- SentencePiece-based tokenization

Examples:

- Mistral Large
 - Mixtral
-

Importance in AI

Tokenizers determine:

- How much text fits into context windows.
 - API pricing based on token usage.
 - Model performance and efficiency.
-

5. Postman API Practice

What is Postman?

Postman is a popular API testing tool used by developers to create, test, and manage APIs.

It allows users to send requests to APIs and view responses without writing code.

Why Use Postman?

- API testing
 - API debugging
 - Authentication testing
 - Response validation
 - Learning REST APIs
-

Common HTTP Methods

GET

Retrieves data.

Example:

Get user information.

POST

Creates new data.

Example:

Create a new user account.

PUT

Updates existing data.

Example:

Update user profile.

DELETE

Deletes data.

Example:

Delete a user account.

API Testing Steps in Postman

Step 1

Open Postman.

Step 2

Select request type.

Example:

GET

Step 3

Enter API URL.

Example:

https://api.example.com/users

Step 4

Add headers if required.

Example:

Authorization Token

Step 5

Send request.

Step 6

View response.

Postman displays:

- Status code
- Response body
- Headers
- Response time

Authentication Testing in Postman

Postman supports:

- API Key
- OAuth 2.0
- JWT Token
- Bearer Token
- Basic Authentication

Developers commonly use Postman to test AI APIs such as OpenAI, Gemini, Claude, and other cloud services.
